

자기소개서

저는 3D 게임의 공간과 시점이 어떻게 화면 위에 구현되는지에 관심을 가지며 게임 개발을 시작했습니다. 처음에는 3D 공간이 화면에 그려지는 과정이 막연했지만, 벡터 구조와 월드·뷰·투영 행렬을 학습하며 공간 데이터가 카메라 시점과 투영 과정을 거쳐 화면에 표현되는 흐름을 이해하게 되었습니다. 이를 계기로 게임 세계를 직접 코드로 구현해보고 싶다는 목표를 갖게 되었고, C++를 학습하며 제가 설계한 로직이 실제 플레이 경험으로 이어지는 과정에 매력을 느끼며 게임 클라이언트 개발자를 목표로 하게 되었습니다.

프로젝트를 진행하며 기술적으로 도전했던 경험 중 하나는 DirectX 기반 파티클 시스템 구현이었습니다. 전투 중 패링, 스파크, 먼지와 같은 파티클이 반복적으로 생성되는 구조였기 때문에 Draw Call 증가와 렌더링 비용을 먼저 고려했습니다. 이에 따라 Instance Buffer 기반의 Particle Instancing 구조를 적용했고, 하나의 Point Instance 클래스를 기반으로 방향, 중력, 속도, 가속도 데이터를 조합해 다양한 파티클을 표현할 수 있도록 설계했습니다. 그 결과 동일한 메쉬를 사용하는 파티클을 하나의 Draw Call로 렌더링할 수 있었고, 새로운 파티클을 추가할 때도 별도의 구조를 늘리기보다 데이터 조합으로 확장할 수 있었습니다. 이 경험을 통해 렌더링 비용을 의식한 자료 구조 선택과 데이터 중심 설계가 실제 성능과 확장성에 직접적인 영향을 준다는 점을 배웠습니다.

Unreal Engine 5 프로젝트에서는 기능을 구현하는 데서 끝나는 것이 아니라, 유지보수와 확장성을 고려해 미리 구조를 설계해야 한다는 점을 가장 크게 느꼈습니다. 초반에는 플레이어 클래스와 보스 클래스에 기능이 빠르게 쌓이면서 공격, 방어, 피격, 처형, 카메라, 이펙트 로직이 서로 강하게 얽히는 문제가 생겼습니다. 기능을 추가할수록 한 곳을 수정하면 다른 행동이 예상치 못하게 영향을 받았고, 새로운 액션을 추가할 때마다 기존 액션과의 연계 조건을 다시 점검해야 했습니다. 이 과정에서 구조적 정리가 필요하다는 점을 느꼈고, 플레이어 입력을 바로 실행하지 않고 Action Request로 변환한 뒤 현재 액션 상태와 캔슬 정책을 검사해 실행하거나 입력 버퍼에 저장하는 구조로 정리했습니다. 또한 장비, 포션, 방어, 피격 반응 같은 기능은 Actor Component로 분리하고, 처형 대상과 공격 대상은 Interface를 통해 상호작용하도록 바꾸었습니다. 이를 통해 단순히 동작하는 코드를 만드는 것보다, 기능이 늘어났을 때 안전하게 확장될 수 있는 구조를 만드는 일이 더 중요하다는 것을 배웠습니다.

또한 LLM과 Agent를 반복 작업, 코드 정리, 예외 상황 점검의 보조 도구로 활용했습니다. 다만 생성된 결과를 그대로 적용하기보다 프로젝트 구조와 의도에 맞는지 직접 검토하고 수정했습니다. 이 경험을 통해 AI를 활용하더라도 최종 설계 판단과 코드 품질에 대한 책임은 개발자에게 있다는 점을 배웠습니다.

협업 프로젝트에서는 시간 정지 스킬을 구현하며 기존 시스템과 팀원의 작업 흐름을 고려해 문제를 해결한 경험이 있습니다. 초기에는 플레이어를 제외한 레이어의 Update를 중단하는 방식으로 접근했지만, 이 방식은 다른 팀원이 사용 중인 레이어 구조를 수정해야 하고 충돌 판정이나 UI처럼 멈추면 안 되는 업데이트까지 차단될 수 있었습니다. 그래서 실제 Update를 막는 대신 Tick 함수에 전달되는 DeltaTime을 0으로 넘겨 변화가 일어나지 않도록 설계를 변경했습니다. 시간 정지의 영향을 받지 않아야 하는 객체만 원래 DeltaTime을 사용하도록 처리함으로써 기존 구조를 크게 바꾸지 않고 기능을 구현할 수 있었고, 이후 예외 객체가 추가되어도 같은 방식으로 확장할 수 있었습니다. 이 경험은 협업 환경에서 기능 구현뿐 아니라 기존 구조와 확장 가능성을 함께 고려해야 한다는 점을 다시 확인시켜 주었습니다.

앞으로도 기능을 빠르게 구현하는 데서 멈추지 않고, 플레이 감각과 코드 구조를 함께 개선하는 개발자가 되고 싶습니다. 구현한 기능이 프로젝트 안에서 오래 유지되고 확장될 수 있는지 고민하며, 협업 과정에서도 읽기 쉽고 변경에 강한 코드를 작성하는 게임 클라이언트 개발자로 성장하겠습니다.